

2026 Cangjie开源社区技术课题

课题分解概览

编号	课题名称
1	诊断报错：编译器诊断质量与 IDE 体验提升
2	CJ-Reduce：仓颉编译器测试的问题用例自动缩减
3	性能优化：二进制体积和冷启动时间优化
4	AI编码辅助：基于DSL的仓颉应用自演进开发框架
5	仓颉 sdk 版本管理工具
6	仓颉中心仓多仓配置与私有化体系建设
7	Read-Eval-Print Loop
8	基于 TIOBE TQI 质量模型的代码度量与工程评估
9	项目级API文档生成及测试

诊断报错：编译器诊断质量与 IDE 体验提升

技术背景

- ❑ 编译器诊断是用户和大模型接触语言工具链的第一入口，直接影响仓颉学习成本、迁移效率、IDE/LSP 使用体验和 AI 修复质量。
- ❑ 仓颉前端已具备 Parser、AST、Sema、类型系统、泛型、模块导入、宏和注解等完整语义链路，诊断不再只是报错，而要解释复杂语义失败的根因。
- ❑ 该方向边界清晰、收益直接、可拆分为小 PR，也是沉淀 LLM 可消费诊断数据的基础入口。

技术挑战

- ❑ 类型推断失败、泛型约束不满足、重载解析失败、宏展开失败、跨包符号不可见等问题涉及多个前端阶段。
- ❑ 部分场景能正确拦截，但缺少候选信息、相关位置、上下文原因和下一步修复建议。
- ❑ IDE 和 LLM agent 都需要结构化诊断信息，单纯文本报错不足以支撑跳转、快速修复、多位置关联展示和自动修复闭环。

当前现状

- ❑ 已有DiagnosticEngine、Parser 诊断、Sema 诊断、JSON formatter、宏调用位置 note 等基础设施。
- ❑ 高频语法错误、泛型/类型推断失败、重载失败、宏展开错误、跨包符号错误等场景仍有较大优化空间。
- ❑ 当前适合从错误样例库、golden tests、结构化诊断字段和 IDE 友好输出开始补强。

技术诉求

- ❑ 建立高频错误诊断样例库，覆盖语法、类型、泛型、模块、宏和注解等核心场景。
- ❑ 为复杂语义失败补充根因解释、候选项、相关位置、修复建议和稳定错误码。
- ❑ 统一结构化诊断输出，支撑 IDE/LSP 展示、跳转、快速修复、LLM 自动修复和自动化回归测试。

目标 & 要求

- ❑ Parser 诊断交付：形成高频语法错误样例集与诊断优化，覆盖缺括号、缺逗号、泛型约束写法错误、lambda/match/try 语法恢复和多错误继续诊断；指标要求是典型错误样例有稳定错误码、主错误位置、至少 1 条可执行修复建议，并通过 golden tests 固化。
- ❑ Sema 诊断交付：形成类型不匹配、泛型实参错误、重载失败、接口实现不完整、跨包可见性错误的诊断优化；指标要求是复杂语义失败能输出 root cause、candidate symbols/types、related spans 和 expected/actual 信息，避免只给泛化失败文本。
- ❑ IDE/AI 诊断交付：扩展 JSON 诊断字段，补充 notes、helps、related spans、fix hint、error category 和稳定错误码；指标要求是 LSP/IDE 能消费结构化位置与候选修复，LLM agent 能基于 JSON 诊断生成可验证修复。
- ❑ 样例库与回归交付：沉淀可维护的诊断样例库、贡献指南和回归测试基线；指标要求是新增/优化诊断均带最小复现、预期输出、验证命令和错误码稳定性说明。

质量要求

- ❑ 原本能编译通过的正向用例仍然通过
- ❑ 原本应编译报错的负向用例仍然编译报错，且报错数量及报错语义不变，允许有信息增强
- ❑ 除了 diagnostic 文件，前端产物（cjo、CHIR、.bc 等）都应和之前保持一致
- ❑ 编译性能不应有明显劣化（<1.05x）

CJ-Reduce：仓颌编译器测试的问题用例自动缩减

技术背景

- ❑ 仓颌前端处理链路覆盖 Parser、AST、Sema、模块管理、宏展开、注解检查、常量求值、增量编译和 CHIR 下沉等阶段，真实问题往往跨阶段触发。
- ❑ Fuzzing、大模型生成代码、真实工程反馈的问题样例通常规模较大，可能涉及多文件、多包、import 依赖、宏定义/调用和构建配置，人工缩减成本高。
- ❑ 业界已有 C-Reduce、C-Vise、Illum-reduce、rust-reduce 等成熟实践，说明自动生成最小复现用例是编译器质量工程的重要基础设施。
- ❑ 补足工程级用例缩减能力，可显著提升前端问题定位效率（无论是人工还是 agent），并将复杂问题持续沉淀为高质量回归测试资产。

技术挑战

- ❑ 缩减后必须保持目标问题可复现，例如同一 crash、同一错误码、同一类诊断报错、同一类产物差异、相同的运行时错误行为等。
- ❑ 工程级样例存在 package/import、声明依赖、类型依赖、泛型约束、接口实现、宏展开等耦合关系。
- ❑ 简单按文本删行容易破坏程序结构，需要遵循语法进行语法感知缩减。
- ❑ 不同问题类型需要可配置的复现判定逻辑和超时隔离，避免缩减过程引入新的无关错误或性能风险。

当前现状

- ❑ 当前复杂问题样例仍以人工缩减为主，尤其是来自真实工程、Fuzzing 和大模型生成代码的问题。
- ❑ 缺口集中在工程级样例自动缩减、复现条件判定、依赖保持和回归测试自动生成。

*相关论文

Zeller & Hildebrandt, "[Simplifying and Isolating Failure-Inducing Input](#)", IEEE TSE, 2002

Regehr et al., "[Test-Case Reduction for C Compiler Bugs](#)", PLDI, 2012

Sun et al., "[Perses: Syntax-Guided Program Reduction](#)", ICSE, 2018

Zhang et al., "[LPR: Large Language Models-Aided Program Reduction](#)", ISSTA, 2024

Xu et al., "[T-Rec: Fine-Grained Language-Agnostic Program Reduction Guided by Lexical Syntax](#)", ACM TOSEM, 2024

技术诉求

- ❑ 适合先以独立工具链方式建设，降低对前端核心语义逻辑的侵入风险。建立仓颌工程级问题用例自动缩减工具，支持从真实项目或多文件样例中生成最小可复现工程。
- ❑ 支持用户提供复现命令，例如 cjpm build、cjc main.cj、增量编译脚本或特定测试命令。
- ❑ 建立 interestingness test 机制，自动判断缩减后的样例是否仍然触发目标问题。
- ❑ 建立语法感知缩减能力，即先理解源码结构，再按完整声明、语句或表达式删除/替换，减少无效中间样例。
- ❑ 自动生成回归测试，包括最小源码、复现命令、预期错误码、crash signature、诊断输出或差分基线。

目标 & 要求

- ❑ 工程输入管理：支持单文件、多文件、cjpm 工程、跨包 import、配置文件和必要资源的归档与复制，支持在隔离目录中进行复现和缩减。
- ❑ 复现判定机制：支持退出码、crash signature、错误码、诊断文本、AST dump、CHIR dump、相同的运行时错误行为等判定条件。每轮缩减都能区分“原问题仍存在”和“引入了无关新问题”。
- ❑ 语法感知缩减：基于仓颌语法结构删除或替换 import、声明、类型定义、函数体、表达式、泛型约束和 extend 块。优先保持语法有效，失败时能回滚该轮变换并记录失败原因。
- ❑ 依赖保持策略：在缩减过程中保留必要的符号定义、接口实现、泛型约束、宏定义、模块导入和构建入口。输出工程能独立执行，且最小复现不依赖原工程隐藏上下文。
- ❑ 回归测试沉淀：将缩减后的最小复现工程自动转成 Parser、Sema、CHIR、诊断或增量编译回归测试。需包含源码、复现命令、预期错误码/签名、诊断基线和归档说明。
- ❑ 工程化能力：提供缩减日志、超时控制、失败分类、可复现命令、环境信息记录和 CI 集成入口。要求是缩减过程可追溯、可中断恢复，并能批量处理来自 Fuzzing、真实工程和 LLM 生成代码的问题样例。

Cangjie 开源社区

性能优化：二进制体积和冷启动时间优化

技术背景

当前仓颉支持移动端应用开发，而二进制体积和冷启动时间是移动应用的关键性能指标。为了提升用户体验，缩减应用下载时间，加快应用启动速度，这两项指标是作为编程语言与编译器用于达成目标的关键竞争力。

技术挑战

- ❑ 仓颉作为应用层高级语言，包含 GC、反射等高级特性，这类特性通常会需要编译器生成大量元信息用于进行运行时精确式 GC 或动态加载和调用。
- ❑ 一般一个动态库的不同用户会有不同诉求，因此作为动态库开发者，无法对一个通用的库进行功能缩减，只能提供全量功能；而对于仅需要这个库部分功能的开发者而言，又会因为冗余的功能影响性能。

当前现状

仓颉二进制 codesize 分布中，部分场景仓颉元信息段可能占用 20%-40% 空间，其中包含 Stack map、反射信息、泛型信息等元数据。对于标准库，为了满足不同场景的需求，不得不提供反射信息以支持反射能力，即使用户不使用也会占用应用的二进制大小。

技术诉求

为仓颉的高级特性提供元数据延迟发射方案，对于反射等场景缩减编译期的元数据生成，延后至运行时生成或采用动态加载方式按需载入。

进一步增强仓颉后端实现，加强虚函数删除、类型信息删除、死代码删除等特性，强化分析 Pass，提升代码优化效果（包含不可达元信息删除能力）。

具体可选择的技术方向包括但不限于：

- ❑ 通过依赖类型等方案增强跨函数优化效果，提升静态分析和 DCE 能力
- ❑ 提供声明级别反射信息控制能力，仅对需要提供反射调用的声明进行元信息生成。
- ❑ 提供注解特性反射相关变量延迟初始化方案，在实际使用场景懒加载。

目标 & 要求

- ❑ **启用反射场景反射元信息缩减**：二进制反射元信息体积缩减 50%（或迁移至懒加载的动态库中）
- ❑ **Codesize 优化**：标准库 -O2 编译场景 Codesize 优化 10%（无性能劣化）或 -Oz 编译场景 Codesize 优化 25%

AI编码辅助：基于DSL的仓颉应用自演进开发框架

技术背景

本课题面向仓颉鸿蒙应用开发场景，探索基于DSL的 Spec-as-Code（规格即代码）方法，通过规格描述DSL定义应用的**组件契约、架构约束、性能指标、交互规范、演化策略**。

在鸿蒙场景下，开发者可以通过 DSL 声明“应用应当达到的性能指标、组件契约和交互规范”这种“规格即代码（Spec-as-Code）”的设计，使AI辅助开发系统不仅理解代码实现，还能够理解应用设计意图与运行约束，实现从：**规格描述 → 代码生成 → 运行反馈 → 规格感知优化的软件持续演化闭环**。

技术挑战

- ❑ **规格漂移 (Spec Drift)**: 随着鸿蒙系统能力和应用需求持续演进，规格描述、代码实现和运行行为之间容易产生偏差。
- ❑ **架构脆弱性**: 鸿蒙应用涉及复杂的 UI 线程调度与资源生命周期，自动化重构若不具备深刻的“架构意识”，极易破坏界面渲染效率，产生性能退化等问题。
- ❑ **推理成本**: 在 SDD / Loop Engineering模式下，如何确保 Spec 的每次演化调整（如调整性能上限）都能在不破坏业务逻辑的前提下，正确映射到代码实现层。

当前现状

目前，基于 DevEco Code 的 SDD 已能实现从规格到代码的转换。然而：

- ❑ **流程是开环的**: 系统完成代码生成后，与运行环境脱节。即便线上出现内存瓶颈，开发者也无法通过修改“规格”来自动驱动代码修复。
- ❑ **缺乏演化上下文**: 现有的 SDD 生成过程是孤立的，系统不具备“从历史演化中学习”的能力。

技术诉求

构建基于DSL的仓颉应用规格感知演化框架。

- ❑ **规格描述与解析能力**: 支持通过DSL描述应用**组件契约、性能约束、架构规则、演化策略**，形成机器可理解的应用规格模型。
- ❑ **规格感知分析能力**: 建立**Spec - Architecture - Code - Runtime**关联关系，实现规格违规检测、影响范围分析、优化建议生成。
- ❑ **AI辅助演化能力**: 结合Spec信息、代码上下文、运行数据驱动SDD/Loop Engineering生成优化方案和代码Patch。

目标 & 要求

提供仓颉自演进应用开发框架，开发框架及规格描述DSL可以使用仓颉语言实现，也可以使用其他语言实现，支持仓颉应用自演进开发，其能力包括：

- ❑ **闭环反馈触发**: 建立性能监测点，当指标超出 Spec 范围时，自动触发演化分析、重构请求。规格违规检测准确率大于90%。
- ❑ **Spec驱动增量演化**: 当Spec或优化策略变化时，能够**自动定位影响代码、生成增量修改方案、保持架构一致性**。目标AI生成Patch有效率大于80%（编译通过、测试通过且满足优化目标）。
- ❑ **应用持续优化能力**: 针对**内存优化、UI性能优化、生命周期问题**的典型场景，实现**问题发现 → 原因分析 → Patch生成 → 验证反馈**的自动演化闭环，典型性能问题自动优化成功率大于70%。

仓颉 sdk 版本管理工具

技术背景

- ❑ 管理 SDK 本身的下载、安装、多版本共存、切换、更新、回退和卸载，让项目和 CI 明确选择使用哪个版本
- ❑ 降低安装和升级成本，解决多个项目依赖不同 SDK 时反复修改环境的问题
- ❑ 竞品对标：rustup (rust), go.mod (go), swiftly (swift)

技术挑战

- ❑ 需要有完善的 sdk 发现、安装、配置、隔离和清理机制
- ❑ 需要满足企业准入规则

当前现状

仓颉当前安装文档仍以下载安装包、解压/安装、执行envsetup为主；更新时需要先卸载旧版本再安装新版本，没有官方的多版本管理器。

技术诉求

- ❑ 支持特定版本 sdk 安装、配置、更新、回退、卸载等
- ❑ 支持多版本并存和切换
- ❑ 支持下载源配置、完整性校验，以满足企业镜像要求

目标 & 要求

交付件：仓颉 sdk 版本管理工具，包含以下功能

- ❑ sdk 版本查询、安装及配置
- ❑ sdk 版本更新、回退
- ❑ sdk 卸载及本地缓存清理
- ❑ 针对当前 terminal 或全局的 sdk 版本切换
- ❑ sdk 完整性校验
- ❑ 下载源配置，默认从官网下载

仓颉中心仓多仓配置与私有化体系建设

技术背景

- ❑ 多仓配置允许配置多个 URL，并允许同时使用多仓产物；私有化体系需提供一键搭建可部署的 registry 的方式，以减小企业仓库构建难度。
- ❑ 支持企业同时消费公共生态和内部包；避免同名依赖混淆；保证依赖来源满足隔离网络部署要求
- ❑ 竞品对标：多 registry 配置 (jfrog/artifactory, maven 提供三方私仓工具)
- ❑ 需要明确私有仓与中心仓的关系，以及多仓配置下的依赖解析与冲突处理机制
- ❑ 需要完善镜像仓/代理仓体系、仓库制品同步策略等
- ❑ 需要有搭建可部署私仓的能力

当前现状

cangjie-repo.toml可配置registry和缓存路径，中心仓 API 公开；中心仓文档已有上传、下载、索引和返回码规格。但功能聚焦官方中心仓，未见私有 registry/代理/镜像部署指南。

技术诉求

- ❑ 提供多仓配置，允许同时从多仓获取产物；同时完善依赖解析和冲突处理算法，确保从多仓获取的产物依旧满足中心仓当前的基本要求（版本归一、高版本优先、依赖无冲突、顶层 replace 等）
- ❑ 提供一键搭建可部署私仓的方式，支持镜像/代理仓，支持仓库间同步

目标 & 要求

交付件 1：cjpm 提供多仓配置能力

- ❑ 允许配置多个仓，在高优先级仓库找不到想要的三方库时，尝试从低优先级仓库找
- ❑ 依赖解析、冲突处理、版本强制替换功能适配多仓机制，确保原有规则不被破坏

交付件 2：私仓搭建工具

- ❑ 提供一键搭建可部署私有中心仓的能力
- ❑ 支持搭建镜像仓、代理仓等
- ❑ 支持仓库间三方库同步

Read-Eval-Print Loop

技术背景

- ❑ REPL 按“读取代码、立即求值、打印结果、继续交互”循环运行，并持续保存变量、函数、类型和导入等会话状态
- ❑ 降低语言学习和 API 探索成本，支持快速验证小段逻辑和数据
- ❑ 竞品对标：jshell (java), Evcxr (rust), Yaegi (go)

技术挑战

- ❑ 支持全量仓颉语法和特性，支持 import 标准库
- ❑ 保存名称、类型和运行时对象状态，处理声明覆盖、删除等
- ❑ 处理异常、死循环、后端崩溃、中断和会话恢复

当前现状

仓颉可通过cjc/cjpm run快速编译运行，官网有在线体验和示例，但目前仅有本地运行验证的方式；目前没有公开的 REPL 工具

技术诉求

- ❑ 基于仓颉语法特性，支持表达式求值、声明持久化、标准库导入等基本 REPL 功能
- ❑ REPL 能力 client 化，提供基础交互、多行输入、历史展示、自动补全、故障中断和会话恢复等客户端能力
- ❑ 支持直接运行单文件脚本

目标 & 要求

交付件：仓颉 REPL 命令行工具，包含如下功能：

- ❑ 支持全量仓颉语法和特性的表达式求值、声明持久化等 REPL 能力
- ❑ 支持导入标准库
- ❑ 支持多行输入、自动补全等输入优化能力
- ❑ 支持命令、声明等历史信息展示
- ❑ 支持异常处理、故障中断、会话恢复等崩溃处理能力

基于 TIOBE TQI 质量指标的仓颉代码度量工具

技术背景

- ❑ 代码度量工具用于评估工程规模、复杂度、可维护性和演进风险；TIOBE 语言榜单的 TQI 质量模型明确将圈复杂度、文件重复率、扇入扇出等数据标明为必要的质量看护指标
- ❑ 让团队能用数据识别高风险模块、评估重构收益、制定质量门禁和跟踪长期演进
- ❑ 竞品对标：cargo test (rust), go test (go), JUnit (java)

技术挑战

- ❑ 基于仓颉代码，借助编译器、工具链能力收集关键指标
- ❑ 基于 TQI 质量模型进行评分
- ❑ 支持门禁和质量平台搭建，持续治理

当前现状

仓颉已有 cjlint、cjcov、cjpm tree、cjpm bench、LSP、profile.build.performance_analysis 符号索引，这些都能提供部分质量数据；但没有统一的 cj metrics 或面向质量平台的代码度量报告

技术诉求

- ❑ **标准化评分**：基于 TIOBE TQI 质量模型中的相关评分标准进行工程评估。
- ❑ **门禁与趋势**：支持 baseline、阈值、趋势对比，让度量服务持续治理，而不是一次性报告。

目标 & 要求

交付件 1：代码度量工具，收集以下 TQI 需要的信息

- ❑ LOC、注释率、函数/类型/API 数量、包依赖扇入扇出、循环依赖
- ❑ 复杂度、函数长度、泛型/宏使用规模、public API 面积、跨语言接口数量

交付件 2：门禁和质量平台搭建，包含以下功能

- ❑ 质量评分门禁
- ❑ 质量看护平台，支持 baseline、历史趋势分析对比

项目级API文档生成及测试

技术背景

- 基于真实项目源码生成结构化 API 文档，或者基于现有文档进行自动化测试
- 文档链的完备与否会直接影响生态可信度
- 竞品对标：cargo doc (rust), go doc (go), javadoc (java)

技术挑战

- 需要明确 doc comment 语法规则
- 需要构建文档管理和测试体系，包括示例代码编译运行、版本一致性、搜索索引等

当前现状

cangjie_runtime 有大量标准库 API 文档；文档源中存在`<!-- verify -->`样例校验标记，说明官方文档生产链具备示例验证基础；但未提供 cjdoc、doc comment 标准和项目级 API JSON。

技术诉求

- 自动化 API 文档生成，包括 API 声明、参数、返回值、示例代码等必要信息
- 自动化文档测试，基于已有文档测试其中示例代码、跳转链接的正确性，并验证与声明的 sdk 版本对应关系

目标 & 要求

交付件 1：文档生成工具

- 对标业界主流竞品（cargo doc, go doc, javadoc 等），生成结构化 API 文档，包括声明、参数、返回值、示例代码等必要信息

交付件 2：文档测试工具，包括：

- 示例代码正确性测试
- sdk 版本对应关系测试
- 索引链接正确性测试

THANKS